



Decision trees

Decision trees are widely used models for classification and regression tasks and are part of supervised machine learning. Essentially, they learn a hierarchy of if/else questions, leading to a decision.

These questions are similar to the questions you might ask in a game of 20 Questions. Imagine you want to distinguish between the following four animals: bears, hawks, penguins, and dolphins.

Your goal is to get to the right answer by asking as few if/else questions as possible. You might start off by asking whether the animal has feathers, a question that narrows down your possible animals to just two. If the answer is "yes," you can ask another question that could help you distinguish between hawks and penguins. For example, you could ask whether the animal can fly. If the animal doesn't have feathers, your possible animal choices are dolphins and bears, and you will need to ask a question to distinguish between these two animals—for example, asking whether the animal has fins.

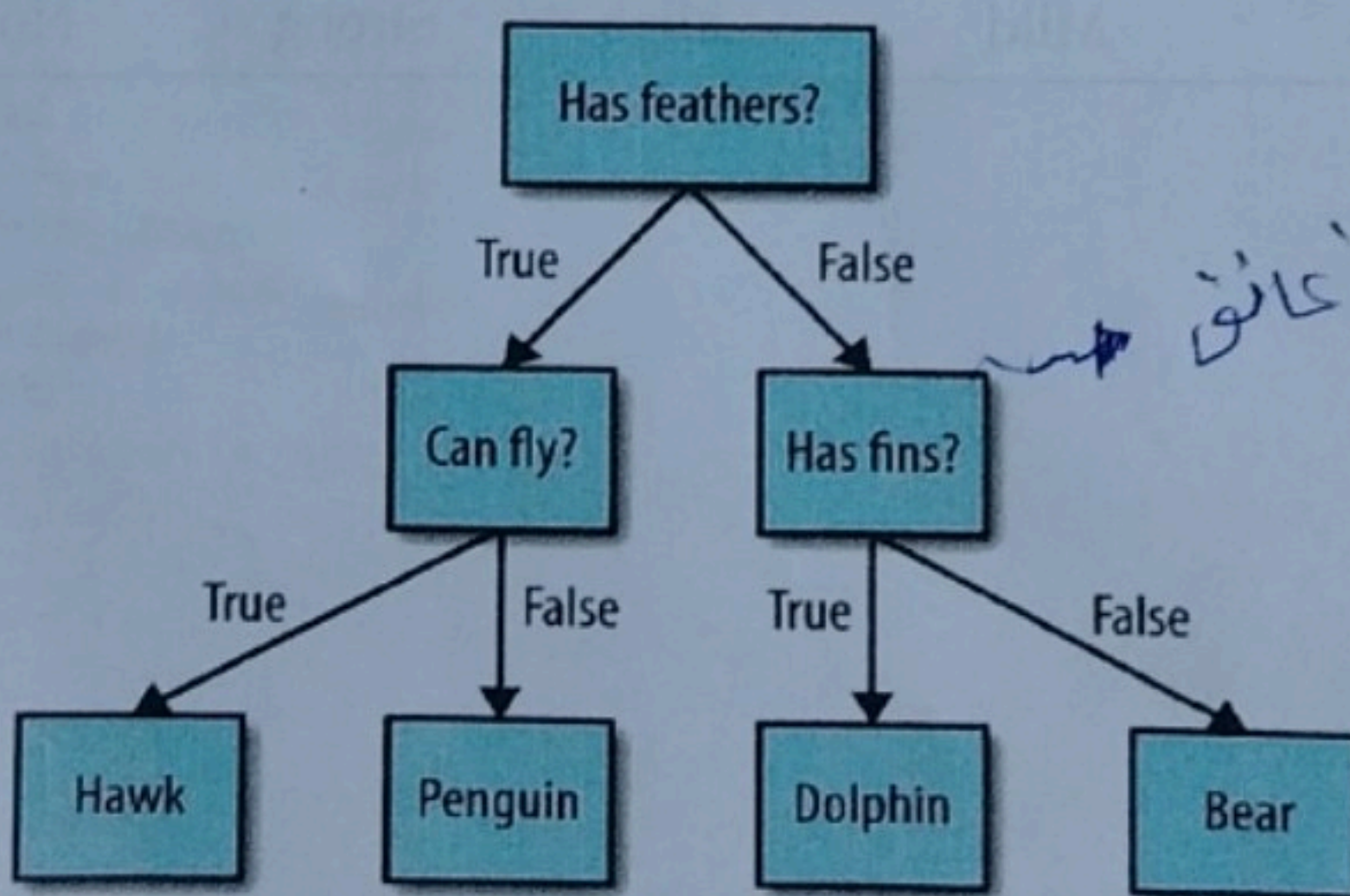


Figure 4-1. A decision tree to distinguish among several animals

In this illustration, each node in the tree either represents a question or a terminal node (also called a *leaf*) that contains the answer. The edges connect the answers to a question with the next question you would ask.

Example: Goal is to Predict When This Player Will Play Tennis?

PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

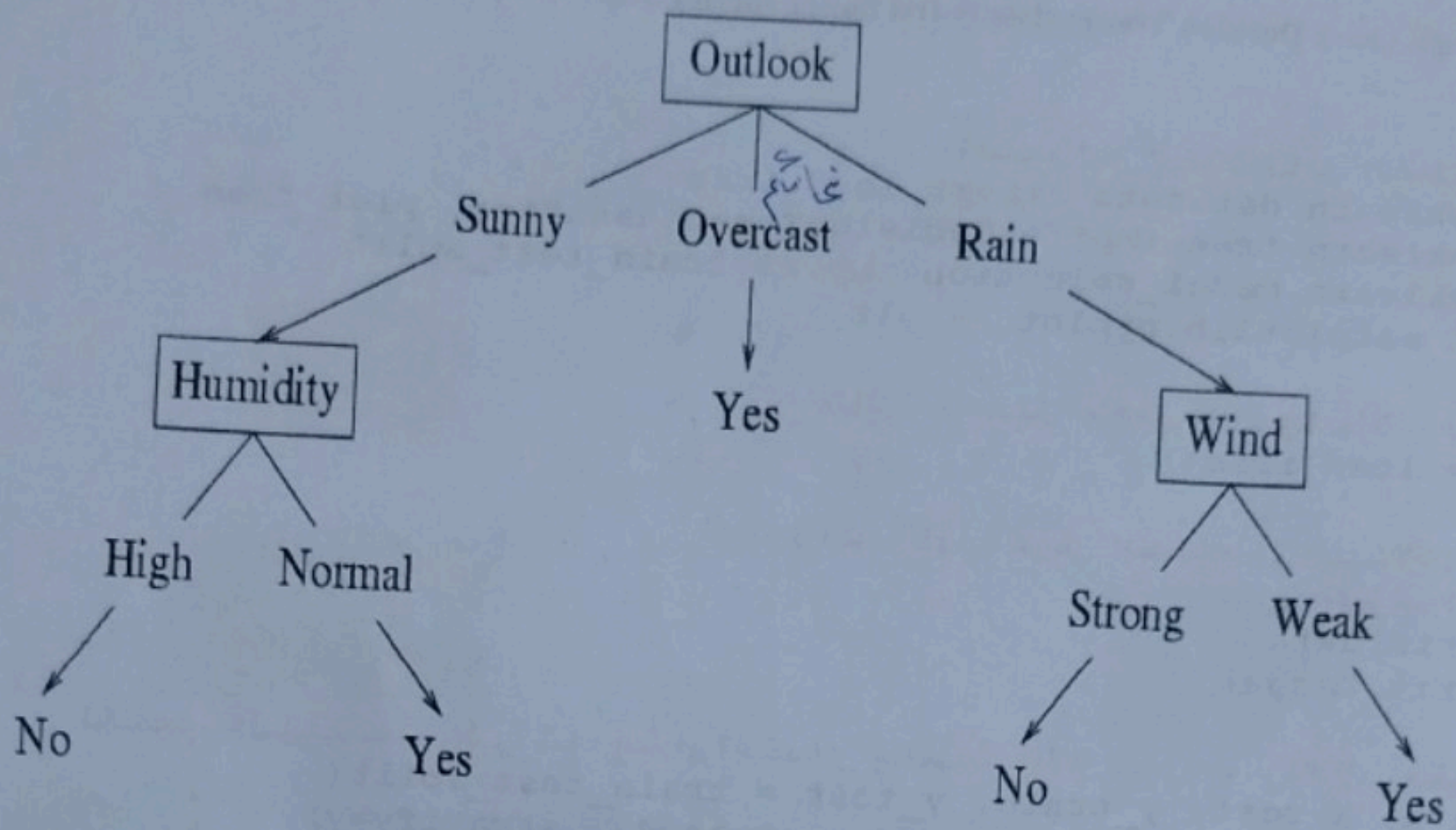


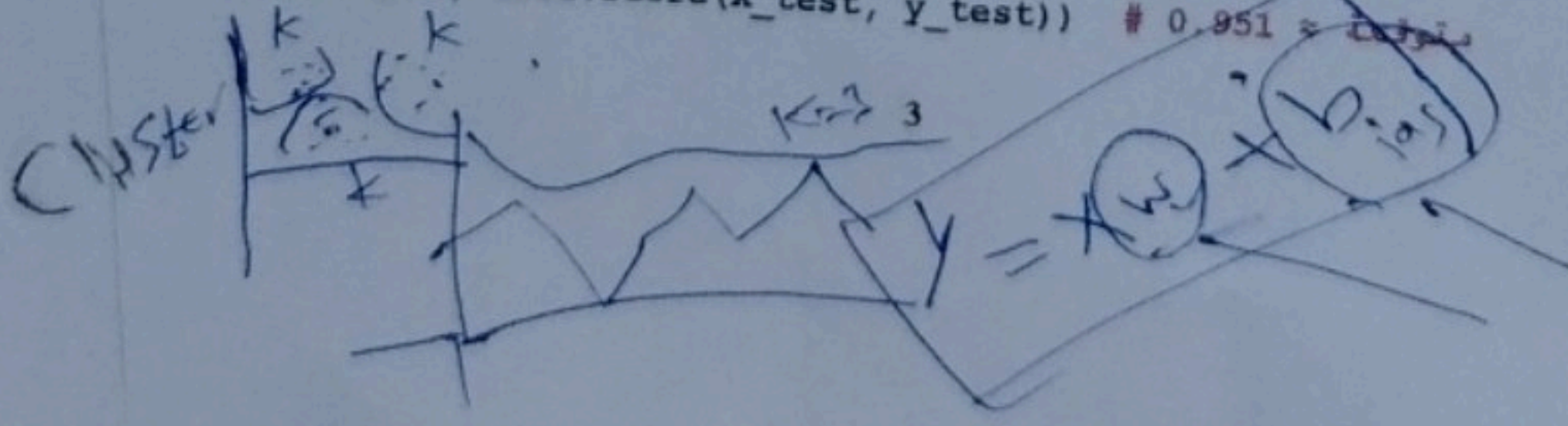
Figure 4.2: A decision tree to Predict When Player Will Play Tennis

In this example, a **Decision Tree** is used to determine whether a breast tumor is **benign or malignant** based on a set of medical measurements.

```

# استيراد مجموعة بيانات سرطان الثدي الجاهزة من sklearn
from sklearn.datasets import load_breast_cancer
# استيراد خوارزمية شجرة القرار (Decision Tree)
from sklearn.tree import DecisionTreeClassifier
# استيراد دالة لتقسيم البيانات إلى مجموعة تدريب واختبار
from sklearn.model_selection import train_test_split
# تحميل مجموعة بيانات سرطان الثدي
cancer = load_breast_cancer()
# تقسيم البيانات إلى تدريب واختبار
# X: الميزات (features)
# y: الفئة (target)
# stratify=cancer.target يعني الحفاظ على نفس نسبة الفئات في التدريب والاختبار
# random_state=42 لضمان نفس النتائج في كل تشغيل
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify=cancer.target, random_state=42)
# إنشاء نموذج شجرة قرار بعمق أقصى = 4 فروع
# (overfitting) العمق المحدود يساعد على منع الإفراط في التعلم
tree = DecisionTreeClassifier(max_depth=4, random_state=0)
# بتدريب النموذج على بيانات التدريب
tree.fit(X_train, y_train)
# حساب دقة النموذج على بيانات التدريب
print("Train accuracy:", tree.score(X_train, y_train)) # 0.988 ≈ متوقعة
# حساب دقة النموذج على بيانات الاختبار
print("Test accuracy:", tree.score(X_test, y_test)) # 0.951 ≈ متوقعة
  
```

حسب كلامها
تكون النتائج
تقسيم البيانات
test (0.951)
train (0.988)
confusion matrix



This example uses a **Decision Tree** to classify **Iris flower species** based on sepal and petal measurements.

```
# استيراد المكتبات المطلوبة
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# تحميل مجموعة بيانات أزهار Iris
iris = load_iris()

# X: (طول/عرض السبلات والبتلات)
# y: نوع الزهرة
X = iris.data
y = iris.target

# تقسيم البيانات إلى تدريب واختبار بنسبة 70% تدريب و30% اختبار
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# إنشاء نموذج شجرة قرار
# للسيطرة على حجم الشجرة ومنع الإفراط في التعلم max_depth نحدد
tree = DecisionTreeClassifier(max_depth=3, random_state=0)

# تدريب النموذج على بيانات التدريب
tree.fit(X_train, y_train)

# تقييم النموذج على بيانات التدريب والاختبار
print("Train accuracy:", tree.score(X_train, y_train)) #0.980
print("Test accuracy:", tree.score(X_test, y_test)) # 0.933

# رسم شجرة القرار
plt.figure(figsize=(14, 8))
plot_tree(
    tree,
    filled=True,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    rounded=True,
    fontsize=10,
    # تلوين العقد حسب الفئة
    # أسماء الميزات
    # أسماء الأنواع
)
plt.title("Decision Tree for Iris Flower Classification", fontsize=14)
plt.show()
```


Decision Tree for Iris Flower Classification

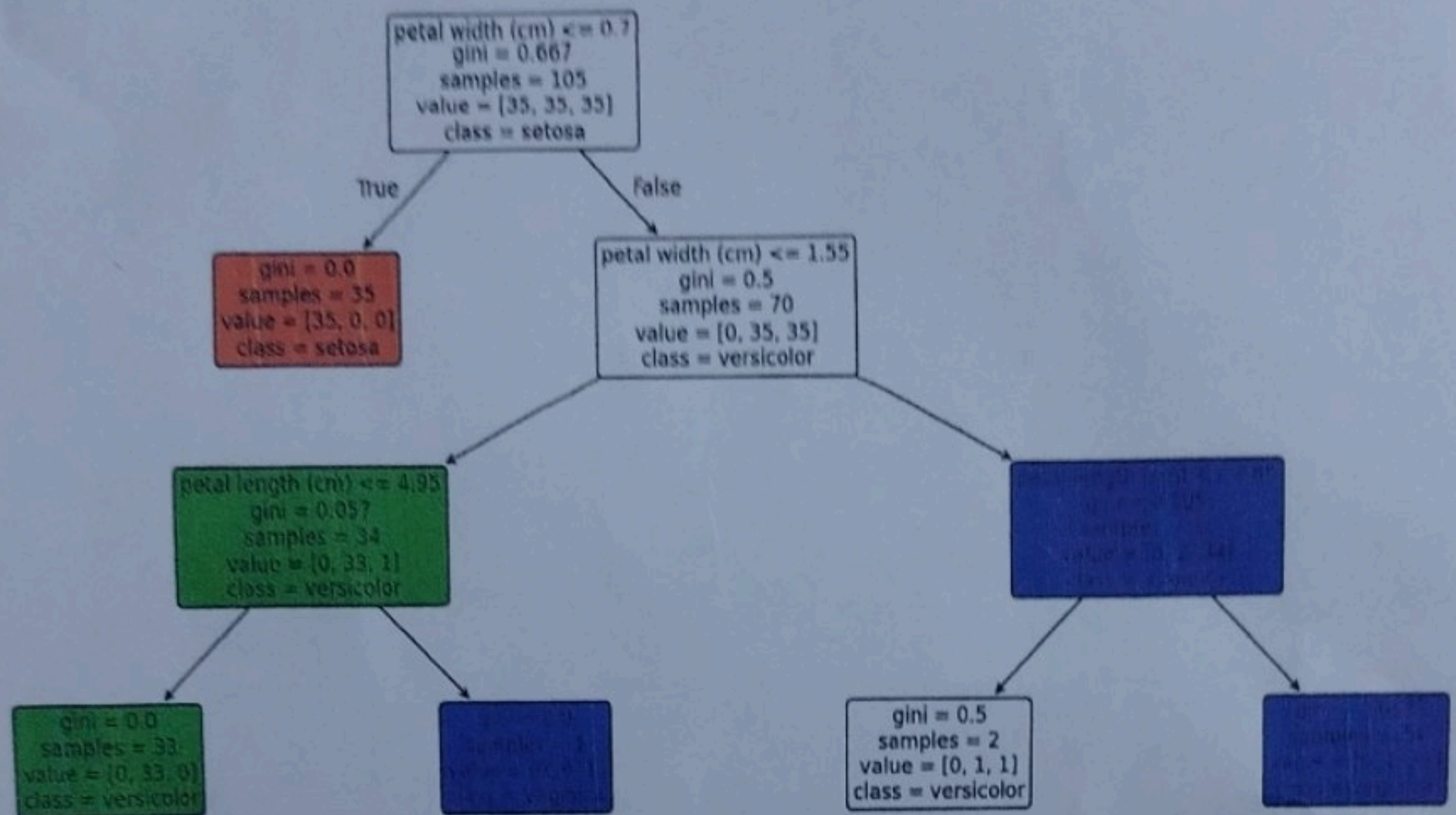


Figure 4.2: The visualization shows how the tree splits the data step by step to reach the correct flower class.